Neural Localization In Non-Linear Stochastic Environment (Theme: 3)

Adyant Srinivasan
dept. of ECE, UCSB
5774443
Team 5
adyant@ucsb.edu

Jash Shah
dept. of ECE, UCSB
A3H4H35
Team 5
jashshah@ucsb.edu

Pratyush Sinha
dept. of ECE, UCSB
A2M1R38
Team 5
psinha@ucsb.edu

***Abstract*—This project introduces a novel approach for implementing the recurrent particle filtering methodology through spiking neural networks (SNNs) with phase-dependent encoding. In this approach, particle weights are dynamically updated using Spike-Timing-Dependent Plasticity (STDP), effectively eliminating the need for re-sampling. We demonstrate that our event-based spiking methodology is capable of efficiently localizing and tracking an object within non-linear dynamic models, even in environments characterized by arbitrary noise. Furthermore, we highlight how this approach enables the network to capture complex relationships, perform online updates, and handle noisy data more effectively than conventional methods.**

***Index Terms*—Particle Filtering, Spiking Neuron Architecture, Localization, Stochastic Noise, Spike Timing Dependent Plasticity**

I. Topic Introduction

Bayesian filters are highly effective in estimating temporal signals within dynamic, noisy environments by leveraging prior observations. A typical filtering approach employs these prior observations to compute posterior state estimations using a known set of stochastic distributions [1]. Particle filtering (PF) is a widely adopted technique in sequential estimation, particularly for dynamic systems such as tracking, robotics, and signal processing. Figure 1 provides a comprehensive overview of the operation of a particle filter, illustrating two iterative time-steps of the typical "predict-update-resample" cycle. However, conventional PF methods often face significant computational challenges due to two primary bottlenecks: 1) the curse of dimensionality and 2) Bayesian weight disparity, which can lead to weight collapse. In contrast, Recurrent Neural Networks (RNNs) are well-suited for learning sequential dependencies in time-series data, making them a promising alternative to traditional particle filtering. Recent developments in Spiking Neural Networks (SNNs) and their biologically inspired learning mechanisms, such as Spike-Timing-Dependent Plasticity (STDP), present new opportunities to enhance the efficiency of RNNs in the context of particle filtering.

In this paper, we propose a novel approach that combines Spike-Timing-Dependent Plasticity (STDP)-based Spiking Neural Networks (SNNs) and phase encoding to implement Recurrent Neural Networks (RNNs) for particle filtering. The proposed method capitalizes on the temporal dynamics and energy efficiency of SNNs, while preserving the flexibility and learning capabilities of RNNs, making it a promising solution for high-performance filtering in dynamic systems. Event-driven neural approaches based on SNNs typically exhibit a tendency to filter out temporally irrelevant intricacies, thereby reducing computational load. However, the discrete dynamics inherent in these methods are limited by their inability to feedback updated weights and integrate resampling mechanisms effectively.

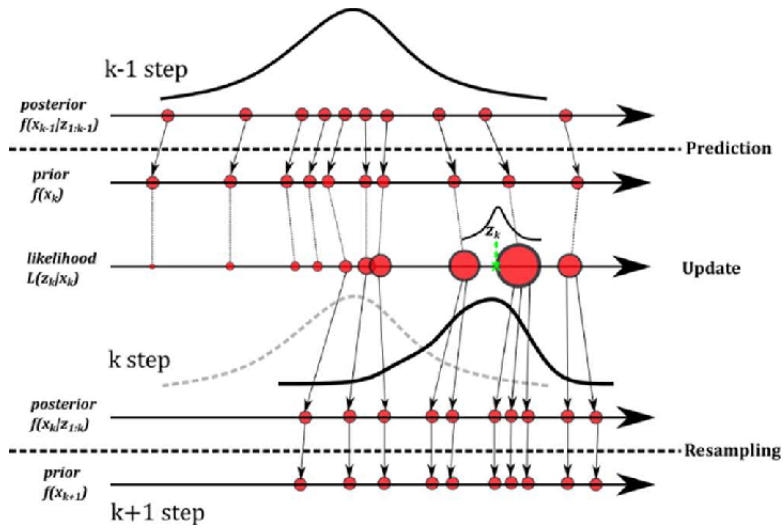

Fig. 1. Schematic Diagram of standard particle filter [4]

II. Related Work

The paper [2] introduces the concept of the Neural Particle Filter (NPF), which is regarded as an enhancement over the traditional particle filter. Nonlinear Bayesian filtering methods, particularly Particle Filters (PFs), are widely employed for real-time dynamic estimation; however, they rely on importance weights, which present significant challenges for neuronal implementation. The proposed NPF provides a weight-less alternative that integrates sensory input and represents posterior probability distributions through samples. This model effectively bridges online state estimation with neuronal network implementation, enabling the capture of tem-

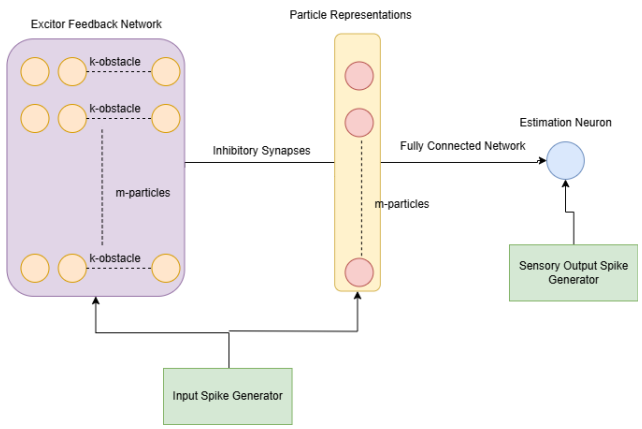

Fig. 2. Proposed Spiking Particle Filter Architecture

poral dynamics and multisensory integration in alignment with Bayesian statistical principles. Additionally, the NPF supports online learning through a maximum likelihood approach.

The authors of paper [3] present an insightful perspective on utilizing data-driven recurrent networks to tackle complex, high-dimensional problems, particularly when dealing with a limited number of particles. Leveraging the powerful learning capabilities of neural networks, they propose a method for approximating posterior distributions more effectively, thereby addressing the "curse of dimensionality" often encountered in traditional particle filtering techniques. This approach enables the Recurrent Particle Filter (Recurrent-PF) to learn complex data relationships without the need for explicit model design, enhancing its versatility across various scenarios. However, it is important to note that this method treats the problem as one of convergence, rather than approaching it as an adversarial problem.

However, both of these methods fail to generalize when exposed to new environments, limiting their applicability. The work presented in [5] extends the use of spiking particle filters (spiking-PF) to various filtering techniques but does not propose a robust algorithm for unconstrained particle filters. In contrast, our approach of simulating particle filters proves particularly effective for handling problems with a large number of variables, where traditional particle filters often struggle due to the requirement for an extensive particle set. Additionally, our model is computationally efficient and can continuously adapt to changing conditions by learning from incoming data, making it highly suitable for a wide range of embedded applications.

III. Proposed Method

The general methodology for our proposed Spiking Particle Filter (Spiking-PF) is outlined in Algorithm 1. Without loss of generality, we define the particle filter in terms of state-space representations as follows:

$$\mathbf{x}(t) = A\mathbf{x}(t) + B\mathbf{u}(t) \tag{1}$$

$$\mathbf{y}(t) = C\mathbf{x}(t) + D\mathbf{u}(t) \tag{2}$$

where $x(t)$ represents the particle state, $u(t)$ represents the input vectors, and $y(t)$ represents the sensory outputs. The estimated output $\widehat{y}$ is used to update the posterior probabilities based on the prior likelihood. In our model, the inputs consist of the known state map, the known input vector, and the noisy sensory readings, with the network generating output predictions for the next state. Typically, there are two stages: one for the update logic and the other for the prediction logic, which are implemented using "phase-encoding" and "Spike-Timing-Dependent Plasticity" (STDP) of neuronal spikes, respectively.

Algorithm 1 SNN Particle Filter Algorithm

Initialize a neurons $N_1 = \{n_1^{(0)}, ..., n_N^{(0)}\}$ with their voltage representing the state space $X_0 = \{x_1^{(0)}, ..., x_N^{(0)}\}$ i.e their location on the map.
for $t = 1$ to T **do**
  for $i = 1$ to N **do**
    Sample Input $N_0^{(t)}$ to update the layer N_1 using motion model $\sim p(x_t|x_i^{(t-1)}, u_t)$.
    Update the LIF rules and phase maps to generate the required number of spikes, representing the estimated output, drawn from the drawn from the prior $p(x_0)$.
  end for
  for Layer 2 **do**
    Inhibit the output spiking based on the output belief, by normalizing the weights: $w_i^{(t)} = \frac{w_i^{(t)}}{\sum_{j=1}^{N} w_j^{(t)}}$
  end for
  Plasticity is used to update prior synaptic network of the output neuron to represent the probabilistic belief of the particles: $\Delta w = \begin{cases} A_+ \exp\left(-\frac{\Delta t}{\tau_+}\right) & \text{if } \Delta t > 0 \\ -A_- \exp\left(\frac{\Delta t}{\tau_-}\right) & \text{if } \Delta t < 0 \end{cases}$
end for

As illustrated in Figure 2, one layer typically represents the particles, which is fully connected to an output predictor neuron. This output predictor neuron, in turn, receives feedback from the sensory outputs. The STDP rules update the weights of the fully connected network, thereby learning the update parameters using the likelihood function and representing the weighted probability for each particle. A spike-generating unit continuously produces input spikes for all the neurons, which are phase-encoded based on their location and updated with excitation from the input vector. Every iteration in the network consists of an "update phase" and a "measure phase" with dynamics mimicing the corresponding phases in a classical particle filtering process.

A network of feedforward and feedback excitatory and inhibitory chains governs the main logic of the predictions required for filtering. Individual neurons are modeled using the Leaky Integrate-and-Fire (LIF) model, where spikes are generated when the synaptic voltage exceeds a predefined threshold. System and input noise are modeled through the LIF's leakage parameter, which is randomized across all

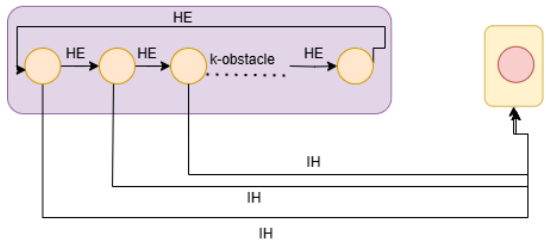

Fig. 3. One dimension of the architecture, representing Spatial Encoding Mechanism

epochs. The following sections discuss the implementation details of our approach:

A. Network Architecture

The implemented system consists of a spiking neural network (SNN), which is composed of excitatory and inhibitory neurons arranged in layers to perform the recurrent updates required for particle filtering. During the "measure" phase of the filtering process, each row of the Excitatory Feedback Network receives u spikes, representing the system input. This input updates the beliefs regarding the particle's location, incorporating the memory of its previous position. In the "update" phase, each neuron in the first two layers receives a constant spiking input from a spike generator, capable of generating n simultaneous spikes to represent the number of possible states in the map. The output layer is inhibited once y spikes are generated, corresponding to the actual sensor response. Based on the belief generated by the network, the STDP rule updates the weights of the fully connected network, driving the system to converge after each epoch (where each epoch consists of one "update" and one "measure" cycle).

B. Spike Generators

The spike generator unit processes both the system input and sensory outputs in the form of a spike train. This spike train encodes the movement of each particle, represented by the voltage levels in the k neurons of any given row in the "Excitatory Feedback Network." For the output neuron, the spike train ensures that the encoding of the post-synaptic spiking activity aligns with the sensor readings. Furthermore, during the update phase, the spike generator produces n spikes, each representing a potential location in the state-space, iterating over all the neurons for every possible state. This phasing guarantees that the updates to the priors and likelihoods remain synchronized and facilitates the formation of "memory" for each state representation, which will be further detailed in III-C.

C. Spatial Encoding

The map was encoded into neuronal representations to efficiently store and update the current state variables. A feedback loop facilitated the "write-back" operation, retrieving the parameters of the previous state. The number of neurons, k, represented the number of sensing beacons on the map, while the membership potential of the N_m^k-th particle indicated the distance of that particle from the k-th obstacle. Additionally, the threshold voltage of the N_m^k-th neuron represented the nearest sensing beacon. As illustrated in Figure 3, each neuron in the Excitatory Feedback Network received spike trains from the spike generators. These neurons had a strong excitatory effect on the next particle, with the feedback loop encoding the location representation and preserving memory after n loops. Furthermore, these neurons exerted an inhibitory effect on the neurons in the subsequent stage, allowing their spike trains to effectively encode the particle's sensing beliefs.

IV. Performance Evaluation

For evaluation purposes, we simulated a particle filtering task in a one-dimensional tracking scenario, where the objective was to estimate the position of a moving object over time, given a set of sensing beacons (e.g., poles). A set of particles was used to represent the state of the object, and the SNN-based RNN propagated the particles' state estimates. The network was trained using noisy observations, with particle weights updated based on the temporal dynamics encoded in the spikes. Figure 4 presents the simulation results, where three beacons (shown in green) and the particles (shown in blue) are plotted. The y-axis represents the belief strength of the particles, while the actual robot position is indicated in red. The graph demonstrates rapid convergence within the first 50 epochs, followed by efficient tracking of the object's position over the subsequent 50 epochs.

An interesting observation was the improvement in time complexity of our model compared to traditional particle filters, primarily due to the elimination of the "resampling" step, which reduced the overall computational overhead. Additionally, our model demonstrated faster convergence with a large population of neurons, outperforming other data-heavy algorithms. The results indicated that the SNN-based RNN significantly enhanced state estimation accuracy, particularly in environments with high noise levels. The system was able to adapt more effectively to changing dynamics, as the STDP learning rule allowed for the refinement of particle weights based on temporal correlations within the data.

Fig 5 further shows how our chosen hyper-parameters were tuned by observing the number of epochs required to converge into 80% confidence intervals. It was observed that the STDP variable A_{max} converged the network faster for a slightly larger values. A similar trend was observed in the STDP weight-decay rate tau as well.

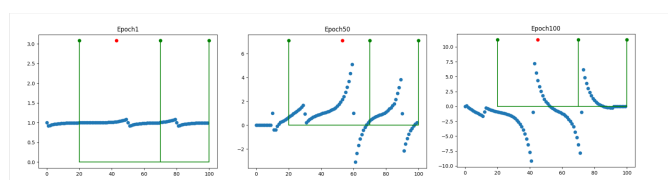

Fig. 4. Results of the proposed particle filtering, showcasing increased weights for actual robot location

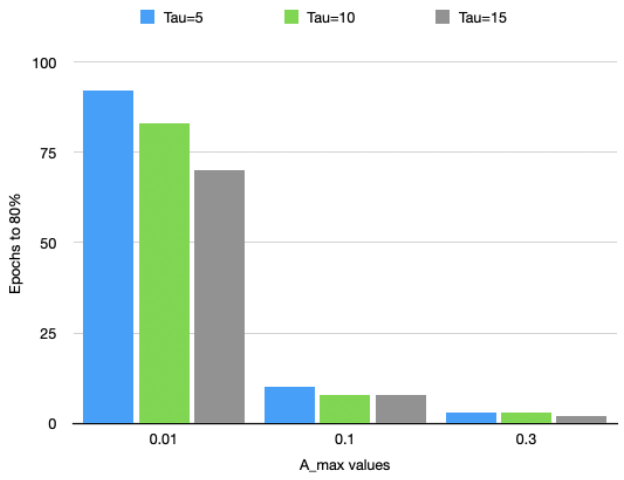

Fig. 5. Number of Epochs to Converge to 80% Confidence Interval

CONCLUSION

This paper presented an innovative approach to implementing particle filtering using recurrent spiking neural networks with STDP and phase encoding. The combination of these techniques enables the network to efficiently learn and propagate state estimates in dynamic systems. Experimental results demonstrated that the proposed approach outperforms traditional particle filters and basic RNN models in terms of both accuracy and computational efficiency. Future work will focus on extending this model to higher-dimensional systems and exploring the integration of more advanced learning mechanisms for even more efficient filtering.

The proposed SNN-filtering methodologies does not fully utilize the temporal advantages of a spiking system, which can furthermore reduce the computational complexity of the system. A systematic approach to eliminate the use of look-up tables can improve the time-complexity manyfold. Furthermore, an extensive research on model-free approaches could adhere better results cultivating from a bias-free learning curve.

REFERENCES

[1] Juárez-Lora A, García-Sebastián LM, Ponce-Ponce VH, Rubio-Espino E, Molina-Lozano H, Sossa H. Implementation of Kalman Filtering with Spiking Neural Networks. Sensors. 2022; 22(22):8845.

[2] Y. Zhang, Z. Qin and F. Wang, ”Particle filters for neural network training,” Proceedings of the 29th Chinese Control Conference, Beijing, China, 2010, pp. 2416-2420.

[3] Ma, X., Karkus, P., Hsu, D., & Lee, W.S. (2019). Particle Filter Recurrent Neural Networks. ArXiv, abs/1905.12885.

[4] Pugalenthi, Karkulali & Park, Hyunseok & Raghavan, Nagarajan. (2020). Piecewise Model-Based Online Prognosis of Lithium-Ion Batteries Using Particle Filters. IEEE Access. PP. 1-1. 10.1109/ACCESS.2020.3017810.

[5] Yamazaki K, Vo-Ho VK, Bulsara D, Le N. Spiking Neural Networks and Their Applications: A Review. Brain Sci. 2022 Jun 30;12(7):863. doi: 10.3390/brainsci12070863. PMID: 35884670; PMCID: PMC9313413.